GPU-Accelerated Two-Dimensional Heat Diffusion with In-Kernel Convergence Detection: From Jacobi and Red-Black SOR to Geometric Multigrid on an NVIDIA Tesla T4

Submitted to Dr. Binod Kumar
Peteti Charantej (G25AIT2026), Aryaveer Rathi (G25AIT2021), Sandeep (G25AIT2036)
GPU Programming Assignment, Problem 1
Implementation: `ipynb` (Google Colab, Tesla T4, CUDA C++)

***Abstract*—We solve the steady two-dimensional heat equation on an $N \times N$ grid with fixed-temperature (Dirichlet) boundaries using the five-point stencil, iterating until the largest pointwise change between consecutive iterates falls below a tolerance ε. Two stationary iterative methods are implemented in CUDA C++ for the Tesla T4: Jacobi Over-Relaxation (JOR, which reduces to the stencil of the problem statement for $\omega = 1$) and Red-Black Successive Over-Relaxation (RB-SOR). Each method has a plain global-memory kernel and a shared-memory tiled kernel. Convergence is detected entirely on the device: every update kernel fuses a warp-shuffle max-reduction and one `atomicMax` per block, and the next iteration's kernels read that value and exit early, so the grid crosses PCIe exactly twice and the host polls only a 4-byte flag every K iterations. At $N = 512$ and $\varepsilon = 10^{-4}$, RB-SOR with the optimal ω^* needs 1,040 iterations against 150,654 for JOR, a 145× reduction, and its iteration count grows as $N^{0.96}$ against $N^{1.46}$ for JOR. We further show that the max-change stopping rule leaves JOR with a 5.29 °C error at $N = 512$, and that single precision cannot support SOR at small ε because round-off is amplified by roughly $1/(2-\omega)$. Finally, we implement a production solver, geometric multigrid (GMG) with red-black Gauss–Seidel smoothing, full-weighting restriction, bilinear prolongation and a full-multigrid start. Each cycle is captured as a CUDA Graph, and the stop decision is made on the device. GMG converges in 3 cycles for every grid from $N = 129$ to $N = 4097$, with a measured convergence factor of 0.043 per V(2,2) cycle, and it stops within 4.5e-07 °C of the exact discrete solution at $N = 1025$. GPU timings in this version are measured on the Colab Tesla T4.*

***Index Terms*—CUDA, heat equation, Jacobi, red-black SOR, geometric multigrid, CUDA Graphs, stencil computation, shared memory tiling, parallel reduction, convergence detection**

I. Introduction

Steady-state heat conduction in a plate is governed by the Laplace equation $\nabla^2 T = 0$. On a uniform grid the five-point discretisation makes every interior temperature the mean of its four neighbours, which leads directly to the update prescribed in the assignment,

$$T_{i,j}^{t+1} = \tfrac{1}{4}\left(T_{i-1,j}^{t} + T_{i+1,j}^{t} + T_{i,j-1}^{t} + T_{i,j+1}^{t}\right), \qquad (1)$$

repeated until $\max_{i,j} |T_{i,j}^{t+1} - T_{i,j}^{t}| < \varepsilon$. Each sweep is perfectly data-parallel and memory-bound, a textbook fit for a GPU, but three practical questions decide whether the GPU version is actually fast: (i) how the global convergence test is evaluated without serialising the device and host every iteration, (ii) whether explicit shared-memory tiling beats the hardware cache for a five-point stencil on a modern GPU, and (iii) whether a better iteration than (1) should be used at all.

Contributions.

- Four CUDA kernels (JOR and RB-SOR, each global and shared-tiled) in one templated program supporting `float` and `double`, arbitrary N, user-supplied initial grids and fixed-temperature boundaries (Section IV).
- A host-sync-free convergence scheme: fused block reduction, per-iteration history on the device, and in-kernel early exit, which makes the stopping iteration exact while the host reads 4 bytes per K iterations.
- Validation that the GPU kernels reproduce a sequential reference bit-for-bit, plus a study of iteration scaling, the relaxation factor, stopping-rule accuracy and floating-point limits (Section VI).
- A bandwidth model of the T4 used to predict and interpret the global-versus-shared comparison.

II. Problem Formulation

Let $\Omega = [0,1]^2$ be discretised [4] by $N \times N$ nodes with spacing $h = 1/(N-1)$. Boundary nodes (i or $j \in \{0, N-1\}$) hold fixed temperatures; in all experiments the top edge is held at 100 °C, left 75 °C, right 50 °C and bottom 0 °C, and the interior starts at 0 °C. Any other boundary set or initial field can be passed as a binary file, whose outer ring is then taken as the fixed boundary. The $n = (N-2)^2$ interior unknowns satisfy $A\mathbf{T} = \mathbf{b}$ with $A = D - L - U$, $D = 4I$, and $\mathbf{b}$ carrying the boundary values.

Stopping rule. The assignment stops when the maximum absolute change $\Delta^k = \|\mathbf{T}^k - \mathbf{T}^{k-1}\|_\infty < \varepsilon$. This is a measure of *progress*, not of *error*: for an iteration with spectral radius ρ one has

$$\|\mathbf{e}^k\|_\infty \lesssim \frac{\rho}{1-\rho}\,\Delta^k, \qquad (2)$$

so a slowly converging method stops far from the solution. Section VI quantifies this.

III. Iterative Methods

A. Jacobi Over-Relaxation

JOR blends the Jacobi value (1) with the old value,

$$T_{i,j}^{k+1} = T_{i,j}^{k} + \omega\Big(\bar{T}_{i,j}^{k} - T_{i,j}^{k}\Big), \qquad \bar{T}_{i,j}^{k} = \tfrac{1}{4}\sum_{\text{nbrs}} T^{k}, \tag{3}$$

and needs two buffers. The Jacobi iteration matrix has eigenvalues $\mu_{pq} = \frac{1}{2}(\cos p\pi h + \cos q\pi h)$, so $\rho_J = \cos \pi h \approx 1 - \pi^2 h^2/2$ and the spectrum is symmetric about zero. JOR maps $\mu \mapsto 1 - \omega + \omega\mu$; with a symmetric spectrum the best choice is $\omega = 1$, and JOR diverges as soon as $\omega > 2/(1+\rho_J) \approx 1 + \pi^2 h^2/4$. Our experiments confirm both statements, so JOR is run with $\omega = 1$, i.e. exactly the stencil of the assignment.

B. Red-Black Gauss–Seidel and SOR

Lexicographic Gauss–Seidel is sequential. Colouring nodes by the parity of $i + j$ makes every red node depend only on black nodes and vice versa, so each half-sweep is fully parallel and the update is done in place:

$$T_{i,j}^{\text{new}} = T_{i,j} + \omega\Big(\bar{T}_{i,j} - T_{i,j}\Big), \quad (i+j) \bmod 2 = c,\ c = 0, 1. \tag{4}$$

The red-black ordering is consistently ordered, so Young's theory applies [1], [2]: $\rho_{GS} = \rho_J^2$ and the optimal factor is

$$\omega^* = \frac{2}{1 + \sqrt{1 - \rho_J^2}} = \frac{2}{1 + \sin(\pi h)}, \qquad \rho_{SOR} = \omega^* - 1. \tag{5}$$

Asymptotically JOR needs $O(N^2)$ iterations to reduce the error by a fixed factor, Gauss–Seidel half as many, and SOR only $O(N)$. With $\omega = 1$ we refer to the method as RB-GS.

IV. GPU Implementation

A. Data layout and thread mapping

Following standard CUDA practice [15], the grid is one row-major array of `real` (template parameter, `float` or `double`). Blocks are $32 \times 8 = 256$ threads, so one warp covers 32 consecutive cells of a row and every global access is coalesced. JOR launches one thread per node; boundary threads do not write, so both ping-pong buffers keep the fixed boundary for free. For red-black, a thread owns the one cell of the active colour in its column pair, $j = 2t_x + ((i+c) \bmod 2)$, so no thread idles and both colours use the same launch grid of $\lceil (N+1)/2/32 \rceil \times \lceil N/8 \rceil$ blocks.

B. Global-memory and shared-memory kernels

The global kernels read the five values straight from global memory and rely on the L1/L2 caches for the four-fold reuse of every value. The tiled JOR kernel stages a $(8+2)\times(32+2)$ tile including a one-cell halo with a cooperative, row-coalesced load, synchronises, and evaluates the stencil from shared memory, so each value is fetched from global memory once per block (plus a 33 % halo overhead). The tiled red-black kernel stages a $(8+2) \times (64+2)$ tile covering 64 columns, since each thread updates one of two adjacent cells. Table I lists the resources reported by `ptxas` (nvcc 13.0, `sm_75`): the global kernels use 18–20 registers, the tiled kernels about twice as many (36–38) for the tile indexing, and at most 5.5 KB of shared memory. Even at 38 registers, four 256-thread blocks (the Turing limit of 1,024 threads per SM) fit, so theoretical occupancy is 100 %.

TABLE I
Kernel resources on sm_75 (ptxas -v, nvcc 13.0)

	float32		float64	
Kernel	Regs	Smem (B)	Regs	Smem (B)
`jor_global`	18	128	20	256
`jor_shared`	37	1488	38	2976
`rb_global`	18	128	20	256
`rb_shared`	36	2768	38	5536

C. Parallel convergence detection

A naïve implementation copies the grid, or a per-block partial max, back to the host every iteration. We instead fuse the test into the update kernels (Algorithm 1):

1) each thread computes $d = |T^{\text{new}} - T^{\text{old}}|$ (0 for idle threads);
2) a warp reduction with `__shfl_down_sync` and a 32-entry shared array give the block maximum;
3) one thread per block issues `atomicMax` (the reduction follows [7]) on the integer view of `hist[k]`. For non-negative IEEE-754 numbers the bit pattern is monotone in the value, so integer max is an exact floating-point max, for 32- and 64-bit types alike;
4) at the start of iteration $k+1$ every block reads `hist[k]`; if it is below ε the kernel returns at once and block $(0,0)$ records k in `convIt`. A kernel that returns early never writes `hist[k+1]`, so the stop state propagates and the grid stops at exactly the first iteration that satisfies the criterion;
5) before the reduction each d is clamped: NaN, Inf and values above 10^{30} become 10^{30}, and a history value of 10^{30} stops the run as *diverged* (a negative `convIt`). Without the clamp a NaN is silently dropped by the max (every comparison with NaN is false), the block maximum becomes 0 and a diverging run is reported as converged. Our first T4 run exposed exactly this: JOR with $\omega = 1.01$ at $N = 128$ "converged" after 37,974 iterations. With the clamp it is reported as diverged, in agreement with the CPU reference.

The host enqueues K iterations asynchronously (default $K = 200$) and then reads the 4-byte `convIt`. At most $K - 1$ kernels are launched after convergence and each exits after one load. The grid is copied host-to-device once and device-to-host once; the history vector (one value per iteration) is copied once at the end for plotting.

```
// exact float/double max through integer atomics (v >= 0)
__device__ void atomicMaxNonNeg(double* p, double v) {
  atomicMax((unsigned long long*)p,
            (unsigned long long)__double_as_longlong(v)); }
```

Algorithm 1 Fused update kernel with convergence test

Require: grid, ω, iteration k, `hist`, `convIt`
1: **if** $k > 1$ **and** `hist`$[k-1] < \varepsilon$ **then**
2: **if** block $= (0,0)$ **and** thread $= 0$ **and** `convIt`$= 0$ **then**
3: `convIt`$\leftarrow k-1$
4: **return**
5: (shared variant) stage tile + halo; `__syncthreads()`
6: $d \leftarrow 0$
7: **if** own cell is interior (and of active colour) **then**
8: $t_{\text{new}} \leftarrow t + \omega(\frac{1}{4}\sum \text{nbrs} - t)$; store; $d \leftarrow |t_{\text{new}} - t|$
9: $d \leftarrow$ WarpMax(d); block max via shared memory
10: thread 0: `atomicMax(hist`$[k]$, d)

D. Performance model

All kernels perform about six floating-point operations per 16 bytes of compulsory traffic (float64), far below the T4 machine balance, so they are bandwidth-bound [8]. Per iteration JOR moves $2sN^2$ bytes (s = bytes per value; one read, one write). Each red-black half-sweep touches every sector of the grid while updating only half of the cells, so RB-SOR moves about $4sN^2$ bytes per iteration, twice JOR. With a sustained DRAM bandwidth of about 230 GB/s, L2-resident working sets (≤ 3 MB of the 4 MB L2) at about 600 GB/s, about 4 µs per kernel launch, and 25 µs per host poll, the model gives a time per iteration and, multiplied by the measured iteration counts, a time to solution. The model predicts that RB-SOR loses a factor of two per iteration but wins by two orders of magnitude overall, and that shared tiling cannot reduce DRAM traffic below the compulsory one-read-one-write, so its benefit on Turing, where L1 and shared memory share the same 96 KB array, should be small (hypothesis H1).

E. Precision

float64 is the default, for accuracy (Section VI-D). It is not cheap on the T4, whose FP64 throughput is 1/32 of FP32. A pure bandwidth argument predicts a factor of two between the precisions; measured, the float32 JOR kernel is 3.5× faster (Section VI), so with the fused 64-bit reduction the float64 kernels are no longer purely bandwidth-bound. float32 is kept as an option.

V. Experimental Setup

All GPU runs use Google Colab with an NVIDIA Tesla T4 (Turing, sm_75, 40 SMs, 16 GB GDDR6, 320 GB/s peak, 4 MB L2, 64 KB shared memory per SM) and `nvcc -O3 -arch=sm_75` [5], [6]; the measured runs used CUDA 13.0 with driver 580.82. Times are taken with CUDA events around the whole solve, including the convergence polls, after a warm-up launch; the program reports iterations, kernel time, end-to-end time, cell updates per second (MLUPS) and effective bandwidth $2sN^2 \cdot \text{iters}/t$. Grid sizes are $N = 64 \ldots 1024$ for all solvers and up to 4096 for RB-SOR; $\varepsilon = 10^{-4}$; $K = 200$.

Validation. The program has a `--verify` mode that runs the same algorithm sequentially on the CPU and compares grids and iteration counts. Because every red-black half-sweep and every Jacobi sweep is order-independent, GPU and CPU follow the same iterates; differences can only come from FMA contraction by `nvcc`. Before deployment the four kernels were additionally executed in a thread-level CPU emulation of CUDA (real barriers for `__syncthreads` and warp shuffles): for $N \in \{20, 21, 37\}$, both precisions, polling interval $K \in \{1, 7\}$, an iteration cap and a user-supplied initial grid, they matched the reference bit-for-bit, including the stopping iteration. Iteration counts and accuracies below are independent of the hardware for this reason.

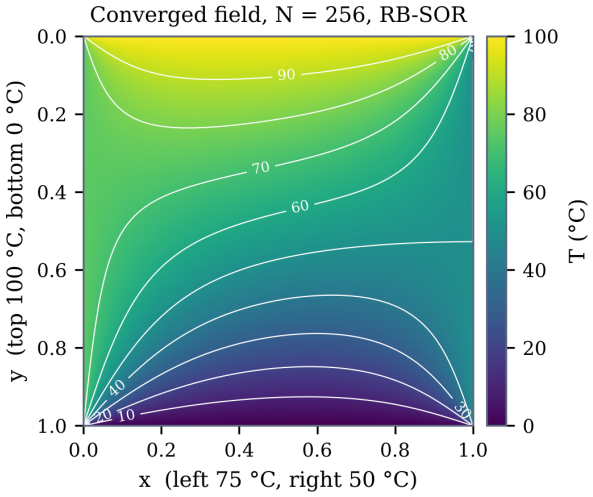


Fig. 1. Converged temperature field (RB-SOR, float64, $\varepsilon = 10^{-4}$) with isotherms.

VI. Results

A. Solution and convergence behaviour

Fig. 1 shows the converged field; the centre value approaches 56.2 °C, close to the analytical superposition value of 56.25 °C for these boundaries. Fig. 2(a) compares the histories of Δ^k at $N = 256$: JOR and RB-GS decay geometrically with rates ρ_J and ρ_J^2, while SOR first oscillates (its error modes are complex with modulus $\omega^* - 1$) and then collapses.

B. Iteration counts and stopping-rule accuracy

Table II and Fig. 3 report the iterations required for $\varepsilon = 10^{-4}$. At $N = 512$, JOR needs 150,654, RB-GS 93,660 (1.61× fewer, close to the theoretical factor of two), and RB-SOR only 1,040. The fitted growth is $N^{1.46}$ for JOR, $N^{1.56}$ for RB-GS and $N^{0.96}$ for RB-SOR. The JOR and GS exponents are below the asymptotic 2 precisely because the stopping rule fires early: (2) with $1 - \rho_J \approx \pi^2/(2N^2)$ allows an error of order $N^2\varepsilon$. The last columns of Table II confirm it: compared with a float64 solution converged to 10^{-10}, JOR stops 5.29 °C away from the discrete solution at $N = 512$ and 21.0 °C at $N = 1024$, whereas RB-SOR stays within 3.4e-03 °C. For slow methods, ε should therefore be scaled with h^2 or replaced by a residual test.

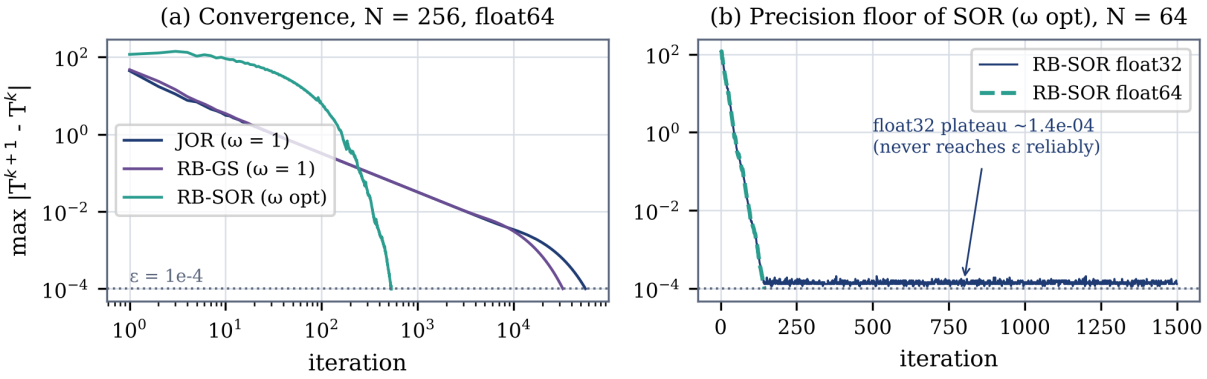


Fig. 2. (a) Maximum change per iteration for the three solvers at $N = 256$. (b) Red-black SOR at ω^*: float64 converges in a few hundred iterations, float32 stalls on a round-off plateau just above ε.

TABLE II
ITERATIONS TO $\varepsilon = 10^{-4}$ (FLOAT64) AND MAX-NORM ERROR VS. THE CONVERGED DISCRETE SOLUTION (°C)

	Iterations to $\varepsilon = 10^{-4}$			Max error vs. exact discrete	
N	JOR	RB-GS	RB-SOR	JOR	RB-SOR
64	5,655	3,107	144	8.0e-2	8.6e-4
128	18,404	10,336	276	0.33	1.6e-3
256	55,830	32,483	531	1.32	2.7e-3
512	150,654	93,660	1,040	5.29	3.4e-3
1024	311,637	228,375	2,035	21.01	4.3e-3
2048	–	–	4,014	–	5.2e-3
4096	–	–	7,869	–	6.8e-3

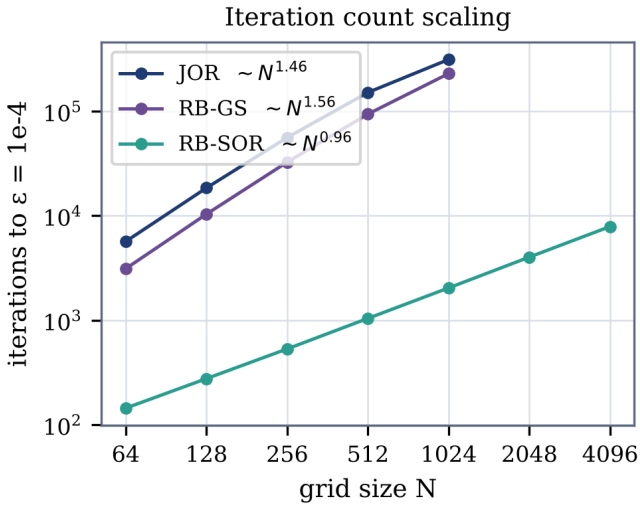


Fig. 3. Iterations to $\varepsilon = 10^{-4}$ versus grid size (log-log) with fitted exponents.

C. Relaxation factor

Fig. 4 sweeps ω at $N = 256$. Iterations fall from 32,483 at $\omega = 1$ to 531 at $\omega = 1.9757$, in agreement with $\omega^* = 2/(1 + \sin \pi h)$, and rise steeply beyond it, the characteristic asymmetric SOR curve: underestimating ω^* is far more expensive than overestimating it. For JOR the sweep

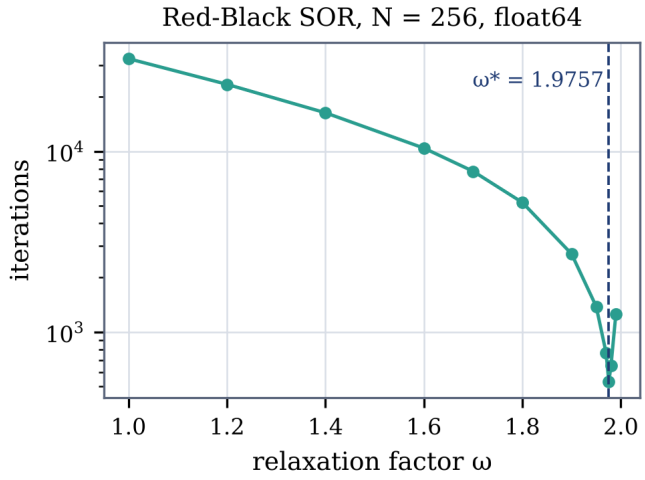


Fig. 4. Effect of ω on red-black SOR; dashed line: theoretical ω^*.

$\omega \in \{0.5, 0.7, 0.9, 1.0\}$ gives monotonically fewer iterations up to $\omega = 1$, and $\omega = 1.01$ diverges, as predicted in Section III-A.

D. Floating-point limits of SOR

In float32, RB-SOR at ω^* reaches a plateau of $\Delta \approx 1.4 \times 10^{-4}$ and does not meet $\varepsilon = 10^{-4}$ (Fig. 2b): at $N = 64$ it crosses the threshold only by chance after about 51 000 iterations, and at $N = 128$ not within 2×10^6. The reason is that a rounding error δ of order $\mathrm{ulp}(T) \approx 7.6 \times 10^{-6}$ near 100 °C is re-injected every sweep and damped only by $\rho_{SOR} = \omega^* - 1$, giving a stationary noise of order $\delta/(1-\rho_{SOR}) = \delta/(2-\omega^*)$, already about 8×10^{-5} at $N = 64$ ($\omega^* = 1.905$) and growing linearly with N. JOR and RB-GS in float32 converge normally. The lesson is general: the attainable tolerance of an over-relaxed method in precision u is about $u \, \|T\|_\infty/(2-\omega)$, so float64 is required for SOR on fine grids.

TABLE III
GPU TIME TO $\varepsilon = 10^{-4}$, FLOAT64

Solver	N	Iter.	Global (ms)	Shared (ms)	Gl./Sh.	GB/s
JOR	64	5,655	28.3	31.4	0.90×	12
JOR	128	18,404	90.7	105.6	0.86×	52
JOR	256	55,830	666.6	730.6	0.91×	86
JOR	512	150,654	6,303.8	7,032.9	0.90×	99
JOR	1024	311,637	56,922	63,586	0.90×	91
RB-GS	64	3,107	26.0	29.7	0.88×	7
RB-GS	128	10,336	77.1	87.5	0.88×	34
RB-GS	256	32,483	493.0	550.6	0.90×	68
RB-GS	512	93,660	4,345.6	4,839.0	0.90×	90
RB-GS	1024	228,375	47,076	54,799	0.86×	81
RB-SOR	64	144	1.4	1.4	0.98×	6
RB-SOR	128	276	2.7	2.8	0.96×	26
RB-SOR	256	531	8.4	9.1	0.92×	65
RB-SOR	512	1,040	45.0	50.3	0.89×	96
RB-SOR	1024	2,035	412.1	478.2	0.86×	83
RB-SOR	2048	4,014	3,201.8	3,715.0	0.86×	84
RB-SOR	4096	7,869	25,383	29,435	0.86×	83

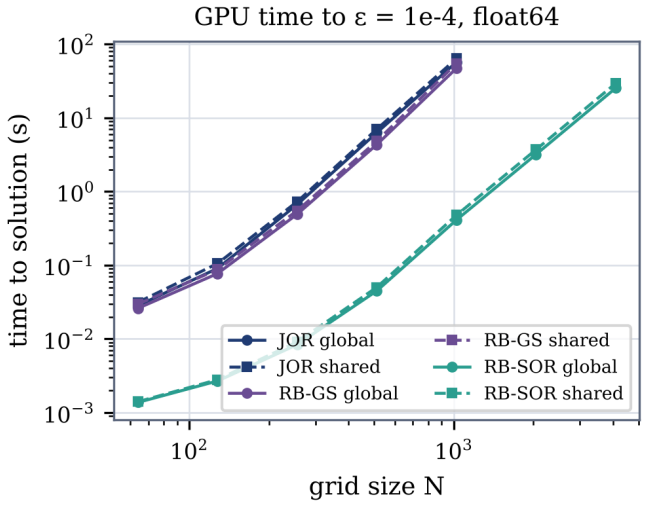


Fig. 5. GPU time to solution versus N; solid: global memory, dashed: shared-memory tiling.

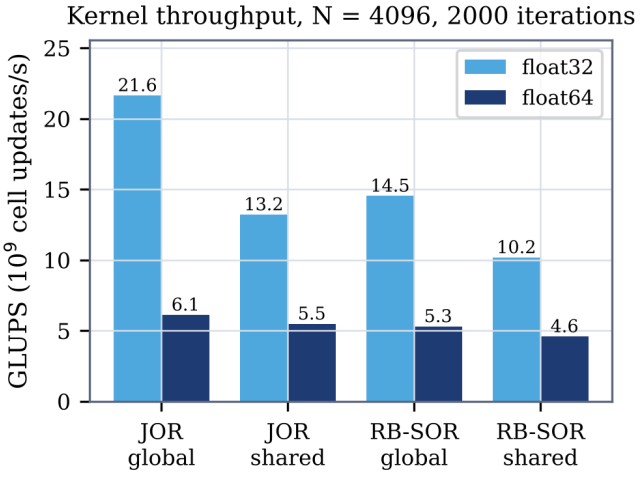


Fig. 6. Throughput of the four kernels at fixed 2000 iterations for the largest grid.

E. GPU performance

Table III and Figs. 5–6 report GPU time to solution and throughput, measured on the Colab Tesla T4. RB-SOR is the fastest solver by a wide margin: at $N = 1024$ JOR needs 56.92 s against 412.1 ms for RB-SOR, a 138× reduction in time to solution even though each SOR iteration costs about twice as much memory traffic. At $N = 4096$ the JOR kernels reach 6.1 GLUPS in float64 and 21.6 GLUPS in float32, i.e. about 98 GB/s of effective bandwidth. That is 98 GB/s in float64 and 173 GB/s in float32 against the 320 GB/s peak: the float32 kernels behave as bandwidth-bound stencils, while float64 is limited by the T4's FP64 rate (3.5× slower instead of 2×). **Shared-memory tiling is slower, not faster.** The global/shared time ratio is 0.86–0.91 for JOR and 0.86–0.90 for RB-SOR in float64 (below 1 favours global memory), so the tiled kernels take 10–16 % longer, and the gap is larger in float32 (Fig. 6). This confirms hypothesis H1 more strongly than predicted: the L1 cache already captures the stencil's reuse, and the tile adds a barrier, a 33 % halo load and twice the registers.

F. Cost of host polling

Fig. 7 varies the polling interval K for RB-SOR at $N = 512$. Polling after every iteration ($K = 1$, the naïve design) takes 57.8 ms; with $K = 200$ it takes 45.6 ms, a 1.27× improvement from removing host synchronisation alone, while the in-kernel early exit keeps the stopping iteration exact for every K. A very large K is not free either: with $K = 1000$ the time rises again to 50.4 ms, because up to $K - 1$ early-exit kernels are queued after convergence and each still costs a launch of a few microseconds. K between 50 and 200 is the sweet spot.

VII. Geometric Multigrid

Section VI showed that even the best stationary method needs $O(N)$ iterations, so its total work grows as $O(N^3)$. Multigrid methods [11]–[13] reduce every error component by a factor that does not depend on N, which gives an $O(N^2)$ solver: the cost of a few sweeps over the grid. This section turns Problem 1 into such a solver.

A. Why smoothing plus coarse grids works

Write the error as a sum of Fourier modes. A Gauss–Seidel sweep replaces each value with the mean of its neighbours, so a mode that changes sign from cell to cell is almost annihilated; for red-black Gauss–Seidel on the five-point Laplacian the smoothing factor is $\mu \approx 0.25$ per sweep [13]. A smooth mode hardly changes, because its neighbours carry nearly the same value: its reduction factor is $1-O(h^2)$, which is the cause of the $O(N^2)$ iteration counts of JOR and RB-GS. A smooth mode, however, becomes oscillatory when it is sampled on a grid with spacing $2h$, $4h$, … (Fig. 8). Multigrid therefore

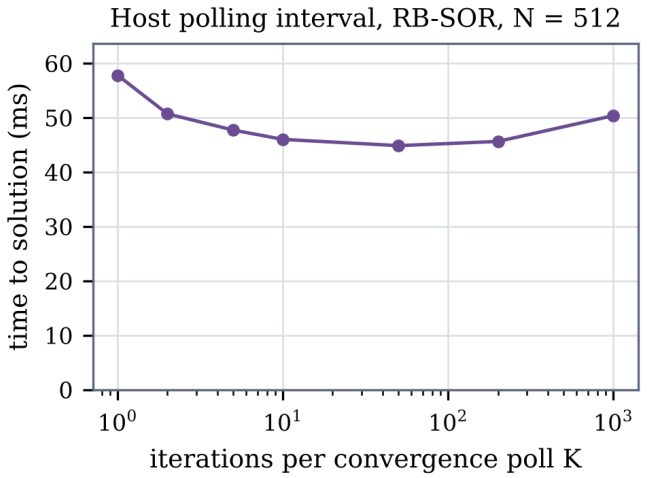


Fig. 7. Time to solution versus the number of iterations between 4-byte host polls.

smooths on the fine grid, moves the remaining smooth error to a coarser grid where it is cheap to remove, and repeats this recursively.

B. Algorithm

Write the problem as $A_h u = f$ with $(A_h u)_{ij} = (4u_{ij} - u_{i\pm1,j} - u_{i,j\pm1})/h^2$, $f = 0$ (an optional uniform source Q gives $f = Q$), and boundary values held in u. The grid hierarchy is $N_0 = N$, $N_{l+1} = (N_l - 1)/2 + 1$, down to 3×3 (one unknown). This requires $N = 2^k + 1$; other sizes stop coarsening early, and the coarsest grid is then solved with $2N_L$ SOR sweeps. On every level we use:

- **Smoother** S: red-black Gauss–Seidel, $u_{ij} \leftarrow \frac{1}{4}(\sum \text{nbrs} + h^2 f_{ij})$, ν_1 sweeps before and ν_2 after the coarse-grid correction (default 2 and 2);
- **Residual** $r = f - A_h u$;
- **Restriction** R: full weighting $\frac{1}{16}[1\,2\,1;\ 2\,4\,2;\ 1\,2\,1]$;
- **Prolongation** P: bilinear interpolation;
- **Coarse operator**: the same five-point stencil with spacing $2h$ (rediscretisation);
- **Coarsest grid**: one red-black sweep is exact on 3×3.

Algorithm 2 gives one cycle; $\gamma = 1$ is the V-cycle and $\gamma = 2$ the W-cycle (Fig. 9). Because level l holds 4^{-l} of the points, a V(ν_1, ν_2) cycle costs about $\frac{4}{3}(\nu_1 + \nu_2 + 2) + 1 \approx 9$ work units (WU), where 1 WU is one red-black sweep of the finest grid and the $+1$ is the convergence statistics. *Full multigrid* (FMG) builds the initial guess: it solves on the coarsest grid, interpolates the solution one level up, applies one V-cycle there, and continues to the finest grid. This costs about one V-cycle and already yields an error comparable to the discretisation error. As before, a cycle counts as one iteration of the assignment's stopping rule.

C. GPU implementation

Every operator is one data-parallel kernel over one level with 32×8 thread blocks: `k_smooth_rb` (one colour, compact mapping as in Section IV), `k_residual`, `k_restrict` (which also zeroes the coarse correction), `k_prolong_add`, `k_fmg_down` and `k_fmg_interp` for FMG, and two bookkeeping kernels. After each cycle `k_stats` computes $\max|u - u_{\text{prev}}|$ and the residual in temperature units, $\max|\sum \text{nbrs} + h^2 f - 4u|/4$, with the warp-shuffle and `atomicMax` reduction of Algorithm 1, and copies u into u_{prev}. A single-thread `k_finalize` writes both values to a history array and sets a device stop flag when the criterion is met, when the cycle budget is used up, or when the values become non-finite. Every kernel begins with `if (stop) return`, so the host can queue several cycles and read the 8-byte control block only every second cycle.

Algorithm 2 Multigrid cycle MG(l) on level l

```
if l is the coarsest level then
    solve A_l u_l = f_l (red-black sweeps); return
u_l ← S^{ν1}(u_l, f_l)                          ▷ pre-smooth
r_l ← f_l − A_l u_l
f_{l+1} ← R r_l;  u_{l+1} ← 0                   ▷ restrict
for g = 1 ... γ do
    MG(l + 1)
u_l ← u_l + P u_{l+1}                           ▷ coarse-grid correction
u_l ← S^{ν2}(u_l, f_l)                          ▷ post-smooth
```

A V-cycle on $N = 1025$ launches about 100 kernels, and most of them work on grids so small that the launch, not the memory traffic, dominates. The host therefore captures one cycle, including statistics and finalisation, into a *CUDA Graph* once and replays it [5]. The graph is valid for all cycles because every kernel argument is a fixed device pointer and the per-cycle state lives in device memory. The grid is copied to the device once and back once. Memory use is $\frac{4}{3} \cdot 3 + 1 = 5$ arrays of N^2 values (670 MB for $N = 4097$ in float64). All kernels use at most 28 registers and 512 B of shared memory (nvcc 13.0). Our T4 model treats each kernel as $\max(\text{bytes}/\text{bandwidth},\ t_{\text{launch}})$ with $t_{\text{launch}} = 2.5\,\mu\text{s}$ per graph node or 4.5 µs per stream launch, using the bandwidths of Section IV-D.

Verification. The program contains a sequential implementation of the same cycle (`--verify`). Run through the thread-level CUDA emulator of Section V (with graph capture emulated by recording and replaying the launches), the GPU code matched it bit-for-bit for V- and W-cycles, with and without FMG and graphs, in both precisions, with a heat source, with a residual criterion, with a truncated hierarchy, with $N = 35$ (a non-$2^k + 1$ size), and with a user-supplied initial grid; the boundary cells were never modified.

D. Results

Table IV and Fig. 10 contain the central result: the number of cycles does not grow with N. V(2,2) with FMG needs 3–3 cycles from $N = 129$ to $N = 4097$, and 6 without FMG, whereas RB-SOR needs 2,033 iterations at $N = 1025$ and JOR 151,037 at $N = 513$. The convergence factor per V(2,2) cycle, 0.043, matches the value of about 0.03–0.05 predicted by local Fourier analysis [13]: one cycle divides the residual

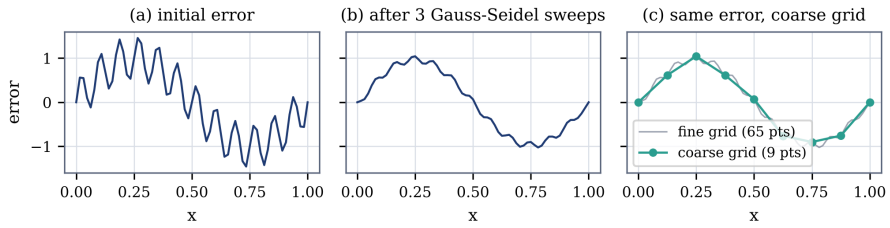


Fig. 8. Smoothing property in 1-D. (a) Error with a smooth and an oscillatory part. (b) Three Gauss–Seidel sweeps remove the oscillatory part, but the smooth part survives. (c) On a grid eight times coarser the same error spans only eight cells and is oscillatory again.

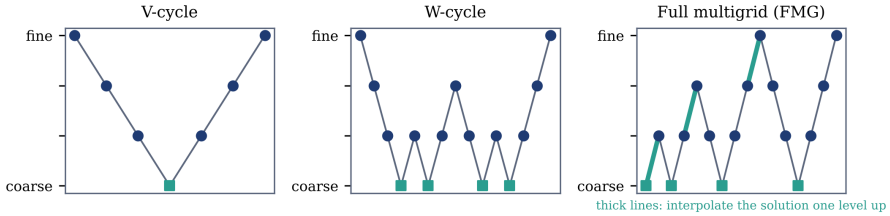


Fig. 9. Order in which the levels are visited (fine at the top). Circles: smoothing on a level; squares: coarsest-grid solve.

TABLE IV
CYCLES TO $\max|\Delta u| < 10^{-4}$ (FLOAT64 UNLESS NOTED) AND CONVERGENCE FACTOR ρ AT $N = 1025$

Variant	129	513	1025	4097	ρ
V(1,1)+FMG	4	4	4	4	0.060
V(2,2)+FMG	3	3	3	3	0.036
W(2,2)+FMG	3	3	3	3	0.031
V(2,2)	6	6	6	6	0.043
V(2,2)+FMG, float32	3	58	400†	400†	–

† not converged within 400 cycles (round-off floor, Section VII-E).

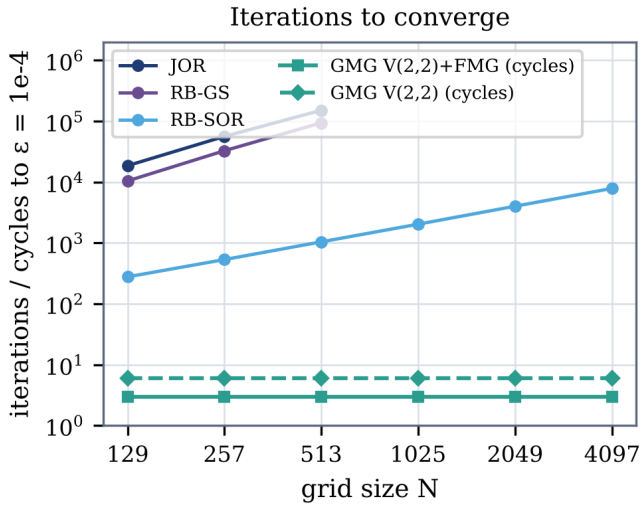


Fig. 10. Iterations (stationary methods) or cycles (GMG) to $\varepsilon = 10^{-4}$, $N = 2^k + 1$.

by about 23 (Fig. 11b). Measured in work units, GMG reaches ε in a few dozen fine-grid sweeps against thousands for RB-SOR (Fig. 11a).

Accuracy. Because each cycle shrinks the error about 23-fold, the max-change criterion is now a reliable error estimate: by (2) the error is at most about $\rho/(1-\rho) \approx 0.04$ times the last change. At $N = 1025$, GMG stops 4.5e-07 °C from the exact discrete solution, against 4.4e-03 °C for RB-SOR and several degrees for JOR.

Tuning. Fig. 12(a) varies the smoothing. More sweeps give a smaller factor per cycle, but the reduction per unit of work is highest for (1,1). Fig. 12(b) truncates the hierarchy at $N = 257$. Stopping at a 129×129 coarsest grid costs 360 WU, because that grid must then be solved by hundreds of SOR sweeps, against 54 WU with the full hierarchy. The coarse levels are cheap in arithmetic but essential for convergence. W-cycles save no cycles here, since the five-point problem is already handled well by V-cycles, and they visit the latency-bound coarse levels many more times.

GPU time. Table V and Fig. 13 report GPU time, measured on the Colab Tesla T4. At $N = 1025$, GMG solves in 8.35 ms against 443 ms for RB-SOR (53×), and the advantage grows linearly with N (213× at $N = 4097$). **CUDA Graphs help only on the smallest grids.** At $N = 129$ the graph changes the time per V-cycle from 0.279 to 0.244 ms (12 % less); from $N = 513$ upward the two differ by at most 2 %. The launches are asynchronous and the CPU enqueues them faster than the GPU runs them, so a cycle is limited mainly by the GPU-side cost of each small kernel, about 3.5 µs for each of the 70 kernels of an $N = 129$ cycle, and only partly by the CPU launch overhead that a graph removes. The effective remedy is fewer coarse-level kernels, for example solving the last few levels inside a single thread block. On large grids the cycle is bandwidth-bound by the finest level, as the five-point smoother was in Section VI.

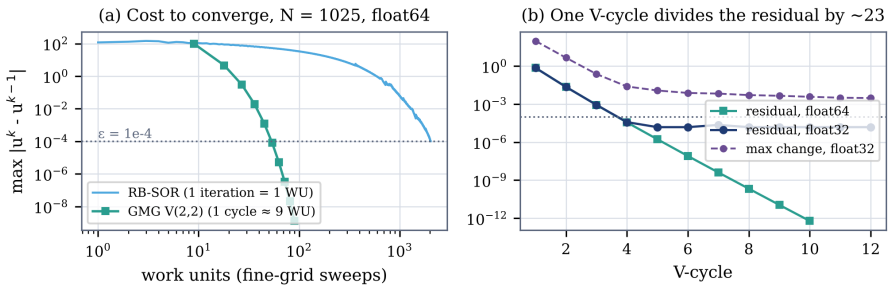


Fig. 11. (a) Max change against work for RB-SOR and GMG at $N = 1025$. (b) Residual per V(2,2) cycle without FMG: float64 falls by a constant factor per cycle down to 10^{-12}; float32 reaches a round-off floor.

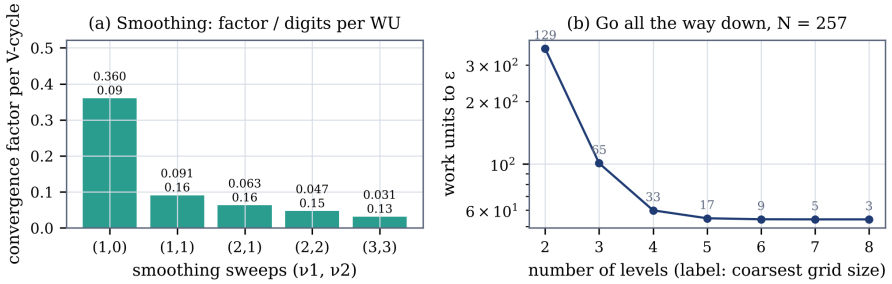


Fig. 12. (a) Convergence factor per V-cycle for different (ν_1, ν_2) at $N = 1025$. Second label line: decimal digits gained per work unit. (b) Work to reach ε against the number of levels at $N = 257$.

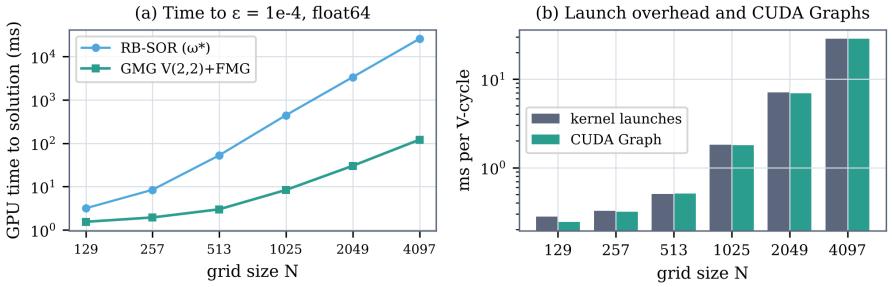


Fig. 13. (a) GPU time to solution. (b) Time per V-cycle with ordinary kernel launches and with a CUDA Graph.

TABLE V
GPU TIME TO $\varepsilon = 10^{-4}$, FLOAT64: RB-SOR (BEST MEMORY VARIANT) VS. GMG V(2,2)+FMG WITH CUDA GRAPH

	RB-SOR		GMG V(2,2)+FMG			
N	iter.	ms	cycles	levels	ms	speed-up
129	278	3.2	3	7	1.54	2×
257	533	8.4	3	8	1.94	4×
513	1,042	52.5	3	9	2.99	18×
1025	2,033	442.7	3	10	8.35	53×
2049	4,015	3,351.8	3	11	30.14	111×
4097	7,881	25,766.5	3	12	120.95	213×

E. Precision

Multigrid removes the $1/(2-\omega)$ amplification that ruined float32 SOR, but a second limit appears: the residual $4u - \sum \text{nbrs}$ is a difference of numbers near 100 °C, so in float32 it carries an absolute rounding error of about $\text{ulp}(100) \approx 7.6 \times 10^{-6}$. That noise drives coarse-grid corrections whose size grows with N. The change between cycles levels off at about 2.6e-04 at $N = 1025$. float32 GMG therefore meets $\varepsilon = 10^{-4}$ only up to $N = 513$, and float64 remains the default. A production extension would keep u and the fine-grid residual in float64 and run the coarse levels in float32 (mixed-precision iterative refinement).

VIII. DISCUSSION

Global versus shared memory. The five-point stencil has a reuse factor of only five and a regular, coalesced access pattern. On Turing the L1 cache and shared memory are the same on-chip array, and neighbouring warps of a block hit in L1/L2 for the north and south rows, so the hardware already captures most of the reuse the tile provides; the tile adds a `__syncthreads`, a 33 % halo load and, for red-

TABLE VI
TOTAL ITERATIONS TO CONVERGENCE ($\varepsilon = 10^{-4}$, FLOAT64) ON THE SAME GRIDS; RB-SOR AT ω^*, GMG V(2,2)+FMG CYCLES

N	JOR	RB-GS	RB-SOR	GMG
129	18,644	10,473	278	3
257	56,166	32,686	533	3
513	151,037	93,923	1,042	3
1025	311,637[a]	228,375[a]	2,033	3
2049	–	–	4,015	3
4097	–	–	7,881	3
× per doubling	2.7–3.0	2.4–3.1	1.96	1
Order	$O(N^2)$	$O(N^2)$	$O(N)$	$O(1)$

[a] at $N = 1024$ (Table II); –: not run (about 10^6 iterations).

TABLE VII
COST COMPARISON ON AN $N \times N$ GRID ($n = N^2$ UNKNOWNS): JOR, RB-SOR AT ω^*, GMG V(2,2)+FMG, WITH MEASURED VALUES

	JOR	RB-SOR	GMG
Iterations to ε	$O(N^2)$	$O(N)$	$O(1)$
Work per iteration	$O(n)$	$O(n)$	$O(n)$, ≈ 9 WU
Total work	$O(n^2)$	$O(n^{1.5})$	$O(n)$, optimal
Memory (grids)	$2N^2$	N^2	$\approx 4N^2$
Parallel depth	$O(1)$	$O(1)$	$O(\log N)$
Measured iterations	151,037[a]	2,033[b]	3 (all N)
GPU time to ε (ms)	–	442.7[b]	8.35[b]
Error at $\varepsilon = 10^{-4}$ (°C)	21.0[c]	4.4e-03[b]	4.5e-07[b]
Tuned parameter	none	ω^*	none

[a] $N = 513$; [b] $N = 1025$; [c] $N = 1024$; –: not run. WU: one fine-grid red-black sweep.

black, staging of a full tile to update only half of it. Explicit tiling pays off for wider stencils (high-order or 3-D) and for temporal blocking, where several sweeps are fused per load [9], [10]. For the five-point update the decisive optimisations are coalescing, fusing the reduction, and above all choosing a better iteration.

Algorithm beats kernel tuning. Kernel-level choices change time by tens of percent at most (shared tiling even costs time); switching from JOR to RB-SOR changes it by two orders of magnitude, and the gap grows linearly with N. Moreover, JOR's result is less accurate at the same ε. Table VI collects the total number of iterations to convergence of all four methods on the same grids, and Table VII summarises the asymptotic cost together with the measured values that confirm it.

Production solver. Geometric multigrid turns the $O(N)$ iteration count of RB-SOR into a constant, needs no tuned relaxation factor, and makes the assignment's stopping rule an honest error estimate. Its GPU difficulty is not bandwidth but latency on the coarse levels. Device-side stopping removes host round trips; on Colab, CUDA Graphs trimmed the remaining launch cost only on the smallest grids (Section VII). For the reader the ranking is simple: JOR is the method of the problem statement and the slowest, RB-SOR is the best single-grid method, and GMG is the method to deploy.

Limitations and extensions. Geometric multigrid as implemented assumes a uniform square grid, constant conductivity and $N = 2^k + 1$. Variable or anisotropic conductivity calls for operator-dependent interpolation, line smoothers or algebraic multigrid (for example AmgX [14]); multigrid as a preconditioner for conjugate gradients adds robustness [3]. Mixed-precision cycles, temporal blocking of the smoother on the finest levels, and agglomerating the coarsest levels onto a single thread block to cut launch latency are the next optimisations.

IX. CONCLUSION

We implemented and evaluated JOR and red-black SOR for 2-D heat diffusion in CUDA on a Tesla T4, each with a global-memory and a shared-memory kernel, with a convergence test that is computed in parallel on the device and costs the host 4 bytes per K iterations. The GPU reproduces the sequential reference exactly. RB-SOR at $\omega^* = 2/(1 + \sin \pi h)$ cuts iterations by 145× at $N = 512$, scales linearly in N, and gives the most accurate answer for a given ε; float64 is required for it. The max-change criterion should be read with (2) in mind, since for slow methods it stops far from the solution. Geometric multigrid is the production answer to Problem 1: 3 cycles for every grid size, an error of 4.5e-07 °C at $N = 1025$, a CUDA Graph per cycle, and the same device-side convergence test and minimal PCIe traffic.

REFERENCES

[1] D. M. Young, *Iterative Solution of Large Linear Systems*. New York, NY, USA: Academic Press, 1971.

[2] R. S. Varga, *Matrix Iterative Analysis*, 2nd ed. Berlin, Germany: Springer, 2000.

[3] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA, USA: SIAM, 2003.

[4] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed. Boston, MA, USA: Brooks/Cole, 2011.

[5] NVIDIA Corporation, "CUDA C++ Programming Guide," 2024. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[6] NVIDIA Corporation, "NVIDIA T4 Tensor Core GPU datasheet," 2019.

[7] M. Harris, "Optimizing parallel reduction in CUDA," NVIDIA Developer Technology, 2007.

[8] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[9] P. Micikevicius, "3D finite difference computation on GPUs using CUDA," in *Proc. 2nd Workshop General Purpose Process. Graph. Process. Units (GPGPU-2)*, 2009, pp. 79–84.

[10] K. Datta *et al.*, "Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures," in *Proc. ACM/IEEE Conf. Supercomputing (SC08)*, 2008, pp. 1–12.

[11] A. Brandt, "Multi-level adaptive solutions to boundary-value problems," *Math. Comput.*, vol. 31, no. 138, pp. 333–390, 1977.

[12] W. L. Briggs, V. E. Henson, and S. F. McCormick, *A Multigrid Tutorial*, 2nd ed. Philadelphia, PA, USA: SIAM, 2000.

[13] U. Trottenberg, C. W. Oosterlee, and A. Schüller, *Multigrid*. London, U.K.: Academic Press, 2001.

[14] M. Naumov *et al.*, "AmgX: A library for GPU accelerated algebraic multigrid and preconditioned iterative methods," *SIAM J. Sci. Comput.*, vol. 37, no. 5, pp. S602–S626, 2015.

[15] D. B. Kirk and W. W. Hwu, *Programming Massively Parallel Processors*, 4th ed. Cambridge, MA, USA: Morgan Kaufmann, 2022.